CHAPTER 10

Testing Elixir

Tests are crucial for gaining confidence that your application works correctly and predictably. Love them or hate them, they remain important and necessary for professional software engineers. Elixir has an excellent built-in testing framework, which we'll cover in this chapter.

In the first chapter, you read that Elixir has a "culture of testing" that's similar to Ruby's culture of testing. But this is the first time we're mentioning tests! This doesn't mean that testing isn't important or should be an afterthought. Instead, it's a byproduct of how this book is structured—the previous chapters focused on how to use the library being shown. This dedicated testing chapter lets us cover techniques to test all of our libraries in one place.

We'll begin by testing our Ecto changesets. These tests won't have database or web requests in them, so they're as simple as they can get. Then, we'll cover other libraries that have been covered in this book: Ecto queries, external API requests, Phoenix requests, and Oban jobs.

By the end of this chapter, you'll be able to read and write Elixir tests. Plus, you'll have patterns to test each of the libraries included in this book. We won't be able to cover everything about testing—there's so much to it! Check out *Testing Elixir [LM21]* by Andrea Leopardi and Jeffrey Mathias for a full book on Elixir testing.

Let's write some tests!

Create Your First Test

Tests can get complex when the code you're testing includes dependencies or complex flows. So, we'll start simple and work our way up to more complex testing cases. But first, let's spend a little time comparing Ruby's testing ideas to Elixir's.